

Roman Domination on Graphs of Bounded Treewidth*

Sheng-Lung Peng and Yuan-Hsiang Tsai

Department of Computer Science and Information Engineering
National Dong Hwa University
Hualien 974, Taiwan

Abstract

A roman domination function on a graph $G = (V, E)$ is a function $f : V \rightarrow \{0, 1, 2\}$ satisfying that every vertex u with $f(u) = 0$ has a neighbor v such that $f(v) = 2$. The weight of a roman domination function is the value $\sum_{v \in V} f(v)$. The roman domination number of a graph G is the minimum weight of all possible roman domination functions on G . In this paper, we show that the roman domination number for a graph with bounded treewidth can be computed in linear time.

1 Introduction

Let $G = (V, E)$ be a finite, simple, and undirected graph. For a vertex $x \in V$, let $N(x) = \{y \in V \mid (x, y) \in E\}$. A vertex subset $D \subseteq V$ is a *dominating set* of G if every vertex which is not in D has a neighbor in D . The DOMINATION PROBLEM for G is to find a minimum dominating set of G . The cardinality of the minimum dominating set is denoted as $\gamma(G)$.

In [3], a new variety of the domination problem called *roman domination* is introduced. A *Roman domination function* on G is a function $f : V \rightarrow \{0, 1, 2\}$ satisfying that every vertex x with $f(x) = 0$ has at least one vertex $y \in N(x)$ such that $f(y) = 2$. By definition, we can partition V into three sets V_0 , V_1 , and V_2 where $V_i = \{v \mid f(v) = i\}$ for $i = 0, 1, 2$. ROMAN DOMINATION PROBLEM for G is to find a Roman dominating function of G such that $2|V_2| + |V_1|$ is the minimum. The minimum value is called *Roman domination number* of G and is denoted by $\gamma_R(G)$. It has been proved that for any graph G , $\gamma_R(G) = \gamma(G) + k$ for any integer k ($2 \leq k \leq \gamma(G)$) [16]. For more background on Roman domination, we refer to [4, 6, 7, 8, 14, 15].

It is known that the Roman domination problem on trees can be solved in linear time and it remains NP-complete when restricted to split graphs, bipartite graphs, and planar graphs [3]. The complexity of the Roman domination problem when restricted to block graphs [9], interval graphs, cographs, graphs with bounded cliquewidth, and AT-free graphs can be solved in polynomial time [13].

2 Preliminary

Some domination-like problems on graphs with bounded treewidth can be solved in linear time, e.g., domination [1], bottleneck domination [11], and power domination [5, 12]. The basic idea for these problems is a dynamic programming approach on a tree-like structure called tree decomposition. In the following, we define and present some terms and concept about this terminology.

Let $G = (V, E)$ be a graph. A *tree decomposition* of G is a pair (T, X) where $T = (I, F)$ is a tree and $X = \{X_i \mid i \in I\}$ is a family of subsets of V , and (T, X) satisfies the following properties.

- $\cup_{i \in I} X_i = V$.
- for every edge $(v, w) \in E$, there is an $i \in I$ with $v \in X_i$ and $w \in X_i$.
- for all $i, j, k \in I$, if j is on the path from i to k in T then $X_i \cap X_k \subseteq X_j$.

The *width* of a tree decomposition $((I, F), \{X_i \mid i \in I\})$ is $\max_{i \in I} |X_i| - 1$. The *treewidth* of a graph G is the minimum width over all possible tree decompositions of G [10].

A tree decomposition (T, X) is called a *nice* tree decomposition if T is rooted and satisfies the following conditions.

- Every node of T has at most two children.

*This work is supported by NSC 95-2221-E-259-010.

- If a node i has two children j and k , then $X_i = X_j = X_k$. In this case, we call i *Join Node*.
- If a node i has one child j , then one of the following holds:
 - *Introduce Node*: $|X_i| = |X_j| + 1$ and $X_i \subset X_j$.
 - *Forget Node*: $|X_i| = |X_j| - 1$ and $X_i \supset X_j$.
- If a node has no child, then it is called *Start Node*.

Consider the graph G depicted in Figure 1 as an example. Figure 2 shows a nice tree decomposition of G .

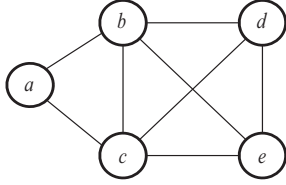


Figure 1: A graph G .

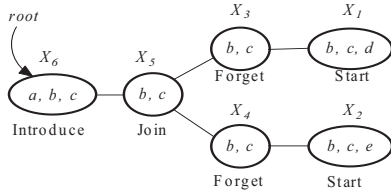


Figure 2: A nice tree decomposition of G .

It was shown in [2] that it can be determined in linear time whether a graph G has treewidth at most k for each constant k . In [10], a nice tree decomposition with treewidth k of G can be constructed given its tree decomposition of treewidth k . In this paper, we propose an algorithm for solving the Roman domination problem on a graph of bounded treewidth given its nice tree decomposition as an input.

3 A linear-time Algorithm

In this section we propose a linear-time algorithm for the Roman domination problem on graphs with bounded treewidth. Let $G = (V, E)$ be a graph with treewidth at most k , and (T, X) is a nice tree decomposition of G . Let T_i denote the subtree of T rooted at node i , X_i denote the vertex subset (or bag) corresponding to node i , and G_i denote the subgraph of G induced by vertices in $\cup_{j \in T_i} X_j$.

For each node i in T , each vertex $x \in X_i$ can be labeled as one number in $\{0^+, 0^-, 1, 2\}$. Number 0^- means that it is not dominated. If one of $N(x)$ is labeled as 2, then x can be labeled as 0^+ . Number 1 means that it can be dominated by itself. A labeling is *legal* if it follows the rules of Roman domination or it can be extended to a solution of Roman domination. For example, a labeling with two vertices x and y such that $(x, y) \in E$ and x (respectively, y) is labeled as 2 (respectively, 0^-) is an illegal labeling. For each bag X_i , we create a table to store all the possible labelings for vertices in X_i . During the process, only legal labelings will be referenced. Beside, we also record the weight for each corresponding labeling. Finally, a labeling at the root with the minimum weight is our solution.

We now describe how to obtain a table of legal labelings for a node i from the tables of its children. We consider the four different cases.

1. **i is a start node:** We first list all the possible labelings for all the vertices in X_i . Then we keep all the legal labelings and compute the weight for each legal labeling.
2. **i is an introduce node:** Let j be its child and $X_i = X_j + x$. We add all possible combinations of x for each labeling in X_j . Remove all the illegal labelings and compute the new weight for each legal labeling.
3. **i is a forget node:** Let j be its child and $X_i = X_j - x$. We copy the table from X_j to X_i and do the following.
 - (a) Remove these illegal labelings. For example, each labeling with x being labeled as 0^- is not a legal labeling.
 - (b) For those legal labelings with the same labels in vertices of $X_j - x$, we keep the one with minimum weight.
4. **i is a join node:** Let j and l be its children. Note that $X_i = X_j = X_l$. We combine the

tables in X_j and X_l into a new table stored in node i by the following steps:

- (a) A labeling with a position labeled as 0^+ in node i can be obtained from a labeling with the same position labeled as 0^- (respectively, 0^+ and 0^+) in node j (the remaining positions must have the same labels) combining a labeling with the same position labeled as 0^+ (respectively, 0^- and 0^+) in node l . For example, a labeling $(0^+, \triangle, \square)$ of i is obtained from combining a labeling $(0^-, \triangle, \square)$ of j and a labeling $(0^+, \triangle, \square)$ of l where $\triangle$ and $\square$ are in $\{0^-, 0^+, 1, 2\}$.
- (b) For the remaining labelings, we obtain them by combining the same labelings in node j and l .
- (c) Compute the new weight for each resulting labeling.

Finally, we find a legal labeling of minimum weight in the root node as our solution.

The following theorem concludes our result.

Theorem 1. *Roman domination problem on graphs with treewidth k can be solved in linear time for every constant $k \geq 1$.*

Proof. The correctness of our algorithm can be obtained easily since we try all possible combinations. In the following we show the time complexity of our algorithm.

1. *Start node:* In this case, it is easy to see that time complexity is bounded by $4^{(k+1)}(k+2)$ since we pick out all possible cases and there are at most $k+1$ vertices in a bag and each vertex has at most 4 labels.
2. *Introduce node:* For the adding vertex, we also choose all possible cases. The time complexity is also bounded by $4^{(k+1)}(k+2)$ for the same reason as Item 1.
3. *Forget node:* It scans all columns in the table for the child and delete columns which is an illegal labeling. The time complexity is also bounded by $4^{(k+1)}(k+2)$ for the same reason as Item 1.
4. *Join node:* It combines two tables stored in the two children of the node. A label 0^+ in a labeling can be obtained from 0^+ and 0^- , 0^- and 0^+ , and 0^+ and 0^+ . So the computation is at most $\sum_{r=0}^s \binom{s}{r} 2^{s-r} 3^r = 5^s$ for s is the

size of the node. Since s is at most $k+1$, the time complexity in this node is $O(5^k)$.

Finally, checking whether a labeling in a bag is legal or not will take $O(k^2)$ time. Since the tree decomposition of G has at most $4n$ nodes [10], the time complexity is bounded by $O(k^2 \times 5^k n)$. $\square$

Consider Figure 2 as an example. In our algorithm, we work from leaves to the root in T . We first consider the bag $X_1 = \{b, c, d\}$, each vertex in X_1 may be labeled as 0^+ , 0^- , 1, or 2. So we create a $4^3 \times (3+1)$ table to store the information of labelings for X_1 as follows.

Table 1: Information of the *start* node of X_1 .

vertices	labels							
b	0^-	0^-	0^+	...	1	1	2	2
c	0^-	0^+	0^+	...	1	1	1	2
d	0^-	0^-	0^-	...	1	2	2	2
weight	0	0	0	...	3	4	5	6

In the *forget* node $X_3 = \{b, c\}$, we first copy the table from X_1 and then refine it. Finally, we obtain a table as follows.

Table 2: Information of the *forget* node of X_3 .

vertices	labels							
b	0^-	0^-	0^+	...	0^+	2	2	
c	0^-	0^+	0^+	...	2	1	2	
d^*	1	1	1	...	0^+	0^+	0^+	
weight	1	1	1	...	2	3	4	

* The information of this row will not be used for the next process.

It is easy to see that the information of X_2 (respectively, X_4) is similar to X_1 (respectively, X_3). As it goes to the *join* node X_5 , the information is shown in Table 3.

Table 3: Information of the *join* node of X_5 .

vertices	labels							
b	0^-	0^+	1	...	0^+	1	2	
c	0^-	0^+	0^+	...	2	2	2	
weight	2*	2	2	...	2	3	4	

* The label of d in X_1 is 1 and the label of e in X_2 is also 1. Thus the combining weight is 2

Finally, let us see the *introduce* node X_6 . The vertex a can be labeled as 0^+ , 0^- , 1, or 2. So the table of X_6 is four times the size of the table of X_5 . The final table of X_6 is shown in Table 4.

Table 4: Information of the *introduce* node of X_6 .

vertices	labels						
a	0^+	0^+	0^+	$\dots$	1	1	2
b	2	0^+	2	$\dots$	2	2	2
c	0^+	2	1	$\dots$	1	2	2
weight	2	2	3	$\dots$	4	5	6

Since X_6 is the root, our algorithm will output 2 as the solution. By a backtracking, we can find the corresponding labeling of (a, b, c, d, e) which is $(0, 2, 0, 0, 0)$.

4 Conclusion

In this paper, we provide a linear-time algorithm to solve the Roman domination problem on graphs with bounded treewidth. Whether the Roman domination problem on permutation graphs can be solved in polynomial time is still open.

References

- [1] N. Betzler, R. Niedermeier, and J. Uhlmann, Tree decompositions of graphs: saving memory in dynamic programming, *Discrete Optimization* 3 (2006) 220–229.
- [2] H. L. Bodlaender, A linear time algorithm for finding tree-decompositions of small treewidth, *SIAM J. Comput.* **25** (1996) 1305–1317.
- [3] E. J. Cockayne, P. A. Jr. Dreyer, S. M., Hedetniemi, and S. T. Hedetniemi, Roman domination in graphs, *Discrete Mathematics* **278** (2004) 11–22.
- [4] P. A. Jr. Dreyer, Applications and variations of domination in graphs, Ph.D. Thesis, Rutgers University, The State University of New Jersey, New Brunswick, NJ, 2000.
- [5] J. Guo, R. Niedermeier, and D. Raible, Improved algorithms and complexity results for power domination in graphs, *FCT 2005*, LNCS **3623**, pp. 172–184, 2005.
- [6] M. A. Henning, A characterization of Roman trees, *Discuss. Math. Graph Theory* **22** (2002) 225–234.
- [7] M. A. Henning, Defending the Roman Empire from multiple attacks, *Discrete Mathematics* **271** (2003) 101–115.
- [8] M. A. Henning and S. T. Hedetniemi, Defending the Roman EmpireVA new strategy, *Discrete Mathematics* **266** (2003) 239–251.
- [9] C.-H. Hsu, C.-S. Liu, and S.-L. Peng, Roman domination on block graphs, *Proceedings of the 22nd Workshop on Combinatorial Mathematics and Computation Theory*, 2005, pp. 188–191.
- [10] T. Kloks, *Treewidth-Computations and Approximations*, LNCS **842**, Springer, Berlin, 1994.
- [11] T. Kloks, D. Kratsch, C.-M. Lee, and J. Liu, Improved bottleneck domination algorithms, *Discrete Applied Mathematics* **154** (2006) 1578–1592.
- [12] J. Kneis, D. Molle, S. Richter, and P. Rossmanith, Parameterized power domination complexity, *Information Processing Letters*, **98** (2006) 145–149.
- [13] M. Liedloff, T. Kloks, J. Liu, S.-L. Peng, Roman domination over some graph classes, *WG 2005*, LNCS **3787**, pp. 103–114, 2005.
- [14] C. S. ReVelle and K. E. Rosing, Defenders imperium Romanum: A clas problem in military strategy, *Amer. Math. Monthly* **107** (2000) 585–594.
- [15] I. Stewart, Defend the Roman Empire, *Scientific American* **281** (1999) 136–139.
- [16] H.-M. Xing, X. Chen, and X.-G. Chen, A note on Roman domination in graphs, *Discrete Mathematics* **306** (2006) 3338–3340.